

End Semestral Examination: Machine Learning 1

M. Tech. (CS) 2024 - 26

Indian Statistical Institute

Maximum Marks: 100, Time: 3 Hrs. 15 Mins.

May 09, 2025

You may attempt ALL the questions. The maximum score you can obtain is 100.

1. Let $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$, where $\mathbf{x}_i \in \mathbb{R}^d$ and $y_i \in \{-1, +1\}$, be a dataset that is linearly separable. Consider the classical perceptron algorithm, which processes one example at a time in a sequential pass over the dataset. The weight vector $\mathbf{w}$ is updated using the following rule:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} + y_i \mathbf{x}_i \quad \text{if } y_i(\mathbf{w}^{(t)} \cdot \mathbf{x}_i) \leq 0,$$

where i is the index of the current data point being examined and t denotes the update step (not necessarily the epoch). Now suppose that we modify the update condition to enforce a margin threshold $\gamma > 0$, such that the perceptron updates only if:

$$y_i(\mathbf{w}^{(t)} \cdot \mathbf{x}_i) \leq \gamma.$$

- (a) Show that if the dataset is linearly separable with margin $\gamma^* > \gamma$, then the modified algorithm converges in a finite number of steps. Derive an upper bound on the number of updates in terms of γ, γ^* , and the maximum norm of the inputs.
- (b) Compare the convergence behavior of this margin-based perceptron to the conventional perceptron algorithm. Under what conditions could the modified perceptron converge faster or slower?
- (c) Now consider a variation of the update rule where the step size decays with time:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} + \eta_t y_i \mathbf{x}_i, \quad \text{where } \eta_t = \frac{1}{t}.$$

Does this version of the algorithm still converge for linearly separable data? Provide a justification or a counterexample.

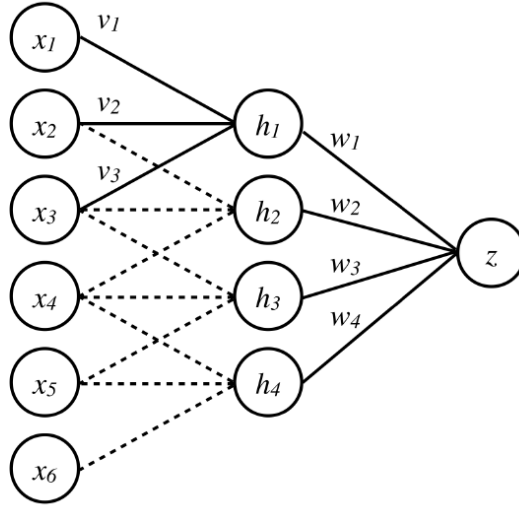
[8 + 6 + 6 = 20]

2. Consider a convolutional neural network (CNN) layer where an input image of size $H \times W$ is passed through a convolutional layer with the following parameters:
- Filter (kernel) size: $K \times K$, Stride: S , Padding: P , Number of filters: F . Assume the input image has C channels (depth), and the convolution is applied across the spatial dimensions (height and width) with padding applied symmetrically on all sides.
- (a) Given an input of size $32 \times 32 \times 3$, a convolutional layer with $F = 16$ filters of size 5×5 , stride $S = 1$, and padding $P = 2$, compute the size of the output feature map.

- (b) Suppose you want to downsample the feature map by a factor of 2 without using pooling. What stride should you use to achieve this, keeping the filter size the same?
- (c) If you stack two such convolutional layers, each with padding $P = 2$, stride $S = 1$, and filter size $K = 5$, what is the overall spatial size of the output relative to the input?

$$[3 + 3 + 4 = 10]$$

3. Consider the following simple 1D convolutional neural network architecture:



- The input is a one-dimensional vector $x = (x_1, x_2, \dots, x_6) \in \mathbb{R}^6$.
- The first layer is a one-dimensional convolutional layer with a **single filter of size 3**, with weights v_1, v_2, v_3 , and **no bias**. The convolution produces 4 outputs:

$$h_i = s \left(\sum_{j=1}^3 v_j x_{i+j-1} \right), \quad \text{for } i = 1, 2, 3, 4,$$

where $s(x) = \frac{1}{1+e^{-x}}$ is the sigmoid activation function.

- The second layer is a **fully connected layer** with weights w_1, w_2, w_3, w_4 , producing a scalar output:

$$z = \sum_{i=1}^4 w_i h_i.$$

- The loss function is the squared error:

$$R = (y - z)^2,$$

where y is the scalar label corresponding to input x .

Note that there are no bias terms in either layer. The output unit is linear (i.e., no activation function is applied after the second layer).

- (a) How many trainable parameters does the network have in total?
- (b) Derive an expression for $\frac{\partial R}{\partial w_i}$ for a given $i \in \{1, 2, 3, 4\}$.

- (c) Express the gradient $\frac{\partial R}{\partial w}$ in vectorized form.
- (d) Derive an expression for $\frac{\partial R}{\partial v_j}$ for $j \in \{1, 2, 3\}$. That is, compute how the loss depends on the convolutional filter weights via the backpropagation chain.

[2 + 5 + 2 + 6 = 15]

4. Given the gradient calculated at a point, the Adam optimizer has three distinct steps. First, update the moving averages. Second, apply the bias correction. Third, update the parameters. Consider the moving average of the square of the gradients. It is given by the recursive formula:

$$S_t = \beta_2 S_{t-1} + (1 - \beta_2) g_t^2.$$

- (a) Derive the expression for S_t only in terms of the gradients $g_0, g_1, \dots, g_t$
- (b) Given your expression in part (a), what is $E[S_t]$ in terms of $E[g_t^2]$ and β_2 ? You may assume that g_i 's are independent and identically distributed.

[4 + 6 = 10]

5. Recall the loss function for k -means clustering with k clusters, sample points $x_1, \dots, x_n$, and centers $\mu_1, \dots, \mu_k$:

$$L = \sum_{j=1}^k \sum_{i \in S_j} \|x_i - \mu_j\|^2,$$

where S_j refers to the set of data points that are closer to μ_j than to any other cluster mean.

- (a) Instead of updating μ_j by computing the mean, let's minimize L with **batch gradient descent** while holding the sets S_j fixed. Derive the update formula for μ_1 with learning rate (step size) ϵ .
- (b) Derive the update formula for μ_1 with **stochastic gradient descent** on a single sample point x_i . Use learning rate ϵ .
- (c) Recall that in the update step of the standard algorithm, we assign each cluster center to be the mean (centroid) of the data points closest to that center. It turns out that a particular choice of the learning rate ϵ (which may be different for each cluster) makes the two algorithms (batch gradient descent and the standard k -means algorithm) have identical update steps. Let's focus on the update for the first cluster, with center μ_1 . Calculate the value of ϵ so that both algorithms perform the same update for μ_1 .

[3 + 3 + 4 = 10]

6. You are given a dataset $\{(x_i, y_i)\}_{i=1}^4$, where $x_i \in \mathbb{R}$, and your goal is to fit a model to predict y from x using **gradient boosting** with squared error loss. You are given the following training data:

$$(x_1, y_1) = (1, 4), \quad (x_2, y_2) = (2, 5), \quad (x_3, y_3) = (3, 8), \quad (x_4, y_4) = (4, 10)$$

and the learning rate is $\eta = 0.5$. At iteration $m = 1$, the base learner $h_1(x)$ is a decision stump that splits the input at $x = 2.5$, assigning a constant prediction to each side of the split.

- (a) Compute the initial prediction $F_0(x)$.
- (b) Compute the residuals at iteration 1, and fit the decision stump $h_1(x)$ based on the split at $x = 2.5$.
- (c) Write the expression for $F_1(x) = F_0(x) + \eta h_1(x)$, and compute $F_1(x_i)$ for all $i = 1, 2, 3, 4$.

- (d) Compute the updated residuals at iteration 2.
- (e) Draw the approximate regression function realized by $F_1(x)$ and also place the training points in the same figure.

[2 + 3 + 4 + 3 + 3 = 15]

7. Consider the data $\left\{\binom{1}{1}, \binom{3}{3}, \binom{2}{*}\right\}$, sampled from a two dimensional(seperable) distribution $p(x_1, x_2) = p(x_1)p(x_2)$, with

$$p(x_1) = \begin{cases} \frac{1}{\theta_1} \exp\left(-\frac{x_1}{\theta_1}\right) & \text{if } x_1 \geq 0 \\ 0 & \text{otherwise,} \end{cases}$$

and

$$p(x_2) \sim U(0, \theta_2) = \begin{cases} \frac{1}{\theta_2} & \text{if } 0 \leq x_2 \leq \theta \\ 0 & \text{otherwise.} \end{cases}$$

As usual, * represents a missing feature value.

- (a) Start with an initial estimate $\theta^0 = \binom{2}{4}$ and analytically calculate $Q(\theta|\theta^0)$ – the **E step** in the EM algorithm. Be sure to consider the normalization of your distribution.
- (b) Find the θ that maximizes your $\theta|\theta^0$ – the **M step**.

[7 + 8 = 15]

8. You are given the predicted scores and true binary labels for a binary classification task. The classifier outputs a score for each instance (a higher score indicates a higher likelihood of class 1), and the true labels are either 0 or 1.

Instance	Predicted Score	True Label
A	0.95	1
B	0.85	0
C	0.75	1
D	0.65	0
E	0.55	1
F	0.45	0

- (a) Use the thresholds $t = \{0.4, 0.5, 0.6, 0.7, 0.8, 0.9\}$ to classify each instance as positive (1) if its score $\geq t$, otherwise negative (0). For each threshold, compute *True Positive Rate* (TPR) and *False Positive Rate* (FPR).
- (b) Approximately plot the ROC curve using the FPR and TPR values obtained for all thresholds.
- (c) Calculate the Area Under the ROC Curve (AUROC).
- (d) Briefly explain what the ROC curve and AUROC value suggest about the classifier's performance.

[3 + 3 + 2 + 2 = 10]

9. A drone is attempting to localize itself along a 1D path using a GPS sensor. At each time step t , the GPS reports a noisy observation $x_t \in \mathbb{R}$ of the true position $\theta \in \mathbb{R}$. The GPS noise is Gaussian: $x_t \mid \theta \sim \mathcal{N}(\theta, \sigma^2)$. The drone has a prior belief about its location based on an earlier map or prediction, modeled as: $\theta \sim \mathcal{N}(\mu_0, \tau^2)$. You are given a sequence of n GPS readings $\{x_1, x_2, \dots, x_n\}$.
- (a) Derive the MAP estimate of the true position θ using the observations and the prior.
 - (b) Interpret the MAP estimate as a weighted average of the prior mean μ_0 and the sample mean of the GPS observations. What do the weights depend on?

[6 + 4 = 10]